MMIA 6013 — IA Generativa y Agentes · Universidad San Francisco de Quito

Enunciado del taller · generado el 2026-10-03

Versión vigente del enunciado. Si trabajas sobre una copia impresa, comprueba la fecha antes de empezar.

Taller 03 v2 (opcional) — Un solver multiagente con GraphRAG

Materia: MMIA 6013 IA Generativa y Agentes **Semana:** 3 · **Sesión:** 15 (sábado, 230 min) y trabajo autónomo **Peso:** 25 % de la nota final, **en lugar del Taller 03**: cada grupo entrega uno de los dos **Fecha de entrega:** la fija el profesor en clase (propuesta: **10 de octubre de 2026, 23:59**), por el canal que se indique en clase **Modalidad:** grupos de 2–3 personas

---

Qué vas a hacer

Vas a construir un sistema de **varios agentes** que recibe el enunciado de una tarea en PDF y la resuelve: lo lee, lo divide en partes, busca lo que cada parte necesita, **escribe y ejecuta el código**, revisa los resultados, corrige lo que falló y redacta el entregable en el formato que pide el enunciado (Markdown, PDF o notebook ejecutado).

Para buscar información usa **GraphRAG**: un grafo que conecta las secciones del enunciado entre sí y con el material del curso.

Dos reglas guían el taller:

1. **Construir → medir → mejorar → volver a medir.**
2. **Una cifra que no salió de una ejecución no es un resultado.** Un reporte bonito con números que nadie calculó no resolvió la tarea.

> **Integridad académica.** Si usas el solver en una tarea de otra materia, **decláralo** y adjunta la traza como evidencia de qué hizo el sistema y qué hiciste tú.

Partes de la carpeta `solver-v2/` (viene con este enunciado, ver «Archivos de este taller»): trae las tareas de práctica, los scripts de la Parte 0, el golden set (`golden_tareas.json`) y el evaluador (`evaluar_solver.py`).

Archivos de este taller

Junto a este enunciado viene la carpeta `taller-03-v2-solver-multiagente/`:

- `solver-v2/` — el kit: las tareas de práctica en `enunciados/` (Markdown y PDF), los tres scripts de la Parte 0 en `parte0/`, el golden set `golden_tareas.json`, el evaluador `evaluar_solver.py`, un cliente mínimo de la H200 (`h200.py`), `verdades.py` (las cifras de referencia de la Tarea C), `requirements.txt` y su `README.md`.
- `notas-teoricas/` — las notas de las cuatro semanas, para la base de conocimiento del grafo.
- `papers/edge-2024-graphrag.pdf` — el paper de GraphRAG.

Empieza con `pip install -r requirements.txt` dentro de `solver-v2/`.

Qué modelo usar

La **H200 de la USFQ** (con VPN GlobalProtect) sirve dos cosas, y necesitas las dos:

- el **LLM**, en `http://172.28.230.10:12555/v1` (réplica en el puerto 12559); el nombre del modelo es el que devuelve `/v1/models`;
- los **embeddings** `bge-m3`, en el Ollama del puerto 11434. (El puerto 12555 no da embeddings.)

Todo el taller cuesta **cero** con la H200. Alternativas: con la clave del curso, `propietario_economico` y `embed_api_economico`; sin red, Ollama local con `nomic-embed-text`.

- **El código que genera el solver corre siempre en tu máquina**, dentro de tu sandbox.
- **No escribas el nombre del modelo de memoria:** léelo del servidor. En el informe pon la ruta, el modelo y la fecha.
- **El modelo de la H200 siempre razona**, y esos tokens salen del mismo cupo que la respuesta. Si se le acaba el cupo, devuelve una respuesta **vacía** con `finish_reason=length`. Tu código debe detectarlo y reintentar.
- **Usa un solo modelo en las Partes 1–2.**

El material

Tareas de práctica (en `solver-v2/enunciados/`):

Tarea	Pide	Entregable	Qué pone a prueba
A — generativo vs. discriminativo	Naive Bayes y regresión logística, curva de aprendizaje, discusión	`reporte.md`, máx. 1 200 palabras	una parte que **depende** de otra
B — temperatura, top-p y entropía	softmax, núcleo, muestreo, pregunta conceptual	notebook **ejecutado**	cifras exactas
C — búsqueda léxica vs. LSA	TF-IDF, LSA, análisis por consulta	`reporte.pdf`, máx. 2 páginas	datos dentro de tablas y límite de páginas

Base de conocimiento. El grafo indexa el enunciado **y** las notas teóricas del curso, para que el solver use las definiciones vistas en clase.

Tus tareas. Además, **al menos dos tareas reales** de la maestría en PDF (de esta u otra materia), con permiso de quien las escribió si no son públicas. Nada que pida datos privados o credenciales.

Parte 0 — Tres errores que no dan error (10 %)

Tres cosas que un solver ingenuo hace mal **sin mostrar ningún error**. Los scripts están en `solver-v2/parte0/`; 0.b y 0.c no usan modelo.

> Si alguno no aparece, algo cambió: di qué.

0.a — El solver que no ejecuta (4 %)

```
python parte0/a_llm_sin_ejecutar.py
```

Le pide al LLM el reporte de la Tarea A **sin dejarle ejecutar nada** y luego corre el experimento real. Con la H200, el reporte traía 30 cifras con aspecto de medidas, y ninguna era real: por ejemplo, Naive Bayes 0,9415 (real: 0,9474) y una curva de aprendizaje inventada que confirmaba justo la tesis del enunciado.

Entregas: tu salida, el modelo usado, y tres frases: ¿por qué no basta con que las cifras estén *cerca*? ¿Qué cifra inventada cambiaría una conclusión? ¿Qué debería comprobar un verificador para detectar esto **sin conocer las respuestas**?

0.b — El RAG que pierde la referencia (3 %)

```
python parte0/b_rag_plano.py
```

Divide la Tarea A en un fragmento por sección y busca lo necesario para responder «*¿con qué datos y qué partición se calcula la curva de aprendizaje?*». Los dos fragmentos que encuentra no dicen cuáles son los datos: eso está en la Parte 1 del enunciado, que no se parece a la pregunta. Si se sigue la conexión «la Parte 3 menciona la Parte 1» en un grafo, se encuentran los 3 datos necesarios (antes, 0).

Entregas: la salida y tres frases: ¿por qué es un fallo de la **búsqueda** y no del modelo? ¿Qué haría un LLM con el primer contexto? ¿Por qué esa conexión se saca con una regla y no se le pide al LLM?

0.c — Terminar sin error no es acertar (3 %)

```
python parte0/c_exit_cero.py
```

Un «programador» de prueba evalúa un árbol de decisión **con los mismos datos con que lo entrenó**: exactitud 1,000, termina sin error y guarda su resultado. Un evaluador que solo mira eso lo **aprueba**. Dos revisiones con código (una del código fuente, otra de si la cifra es creíble) lo **rechazan**.

Entregas: la salida y dos frases: ¿qué otra fuga de datos **no** detectaría la revisión del código? (piensa en un escalador ajustado con todos los datos) ¿Por qué se puede dejar la decisión final a un LLM revisor si las revisiones con código van **antes**?

Parte 1 — El solver base (35 %)

Los agentes mínimos

Un orquestador y **al menos cinco roles**. Los nombres son tuyos; las tareas, no:

Rol	Qué hace	Qué revisa tu código
Lector	PDF → texto por página, con tablas	texto pegado o partido, PDF escaneado
Indexador (GraphRAG)	arma el grafo del enunciado y del material	secciones y dependencias
Planificador	divide en subtareas con tipo y dependencias	que el plan sea válido y cubra todo
Investigador	junta el contexto de cada subtarea	que vayan la sección y sus dependencias
Programador	escribe un script por subtarea de cálculo	el filtro del sandbox
Ejecutor	corre el script en el sandbox	(es código, sin LLM)
Revisor	aprueba o devuelve con la corrección	errores, resultados, NaN, figuras, fugas
Redactor	escribe el entregable en el formato pedido	secciones, extensión, origen de cada cifra

El orquestador es un **grafo de estados**: LangGraph (`langgraph==1.2.12`) o tu propio bucle con las mismas conexiones (programar → ejecutar → revisar → volver a programar, hasta aprobar o llegar al máximo de intentos; y pasar a redactar cuando se acaban las subtareas o el presupuesto).

Las seis reglas obligatorias

Señálalas en tu código:

1. **El plan se valida con código:** ids únicos, dependencias que existen y sin ciclos, y cada sección del enunciado cubierta. Si falla, vuelve al planificador con la lista de problemas.
2. **GraphRAG con base fija.** Las secciones son nodos y cada referencia («Parte 1», «la pregunta anterior») es una conexión `depende_de` que se extrae **con una regla**, no con el LLM. Encima van las entidades que extrae el LLM (datasets, métodos, métricas, conceptos), unidas con las del material del curso. El investigador entrega a cada subtarea **su sección literal y las de sus dependencias**, cada fragmento con su cita.
3. **El código corre en un sandbox:** se revisa antes de ejecutar (sin red, sin procesos, sin borrar, sin `eval`, sin salir de la carpeta); se ejecuta **sin variables de entorno** (ninguna clave llega al script), en su propia carpeta y con tiempo máximo que mata todos sus procesos.
4. **Resultados en un archivo fijo y revisión con código primero.** Cada script guarda sus cifras en un JSON con nombre fijo. El revisor corre primero las revisiones de la 0.c y solo si pasan consulta al LLM.
5. **Origen de cada cifra.** Antes de entregar, el código comprueba que **cada cifra** del entregable salió de una ejecución o está en el enunciado. Las que no, vuelven al redactor.
6. **Siempre se guarda la traza**, también si algo falla: por llamada, **el agente, el modelo, los tokens de entrada y salida, la latencia y el error**; por ejecución, el código de salida, la duración y los archivos creados.

Y el entregable **en el formato pedido**: un notebook se entrega **ejecutado**; un PDF respeta su límite de páginas.

El formato que debe cumplir tu solver

```
resultado = Solver().solve("tarea.pdf", "carpeta_salida/")
# resultado es un dict con:
#   "status":      "completado" | "parcial" | "fallido",
#   "entregables": ["carpeta_salida/reporte.md", ...],
#   "subtareas":   [{"id": ..., "tipo": ..., "status": ..., "intentos": ...}],
#   "usage":       {"tokens_entrada": ..., "tokens_salida": ...},
#   "model":       "nombre del modelo",
#   "trace":       [...]
```

Para el Taller 4, la misma clase tiene además `.run(pregunta)`, donde la pregunta es la ruta a un enunciado, y devuelve un dict con `answer`, `trace`, `status`, `model` y `usage`.

Entregas: el código; el **diagrama del orquestador**; el **grafo de la Tarea A dibujado**, con las conexiones `depende_de` visibles; el plan de una tarea; el modelo y la fecha; y la traza de **una corrida** en la que el revisor haya rechazado al menos un script.

Parte 2 — Evaluación (30 %)

2.a — El golden set (10 %)

Parte de `golden_tareas.json` (las tareas de práctica) y añade **tus dos tareas reales**, cada una con **al menos cinco comprobaciones**:

- de forma: el archivo, las secciones en orden, el límite de palabras o páginas, el notebook ejecutado;
- **al menos dos de tipo `cifra`** con su `verdad_py`: el código que calcula la respuesta correcta. **La respuesta correcta no se escribe a mano: se calcula;**
- **una de `procedencia`**.

Entre todas tus tareas debe haber un notebook, un PDF y **una trampa**: una regla del enunciado que un solver descuidado rompería (no tocar el conjunto de prueba, usar una semilla, un dataset que habría que descargar).

Los tipos de comprobación que entiende el evaluador:

Tipo	Aprueba si
`archivo`	existe el archivo (`patron`, con `minimo` si pide varios)
`secciones`	están todos los encabezados de `lista` (en orden si `en_orden`)
`palabras_max` / `paginas_max`	el entregable no pasa de `valor`
`cifra`	el valor de `verdad_py`, dentro de `tolerancia`, aparece en los 160 caracteres que siguen a `etiqueta`
`cifra_presente`	ese valor aparece en cualquier lugar del entregable
`contiene`	están todos los textos de `lista`
`ipynb_ejecutado`	todas las celdas se ejecutaron y ninguna dio error
`procedencia`	al menos `minimo` de las cifras con dos o más decimales aparece en algo que escribió una ejecución

`verdad_py` es código que deja el valor en la variable `verdad`. Ejemplos:

```
{"id": "A03", "tipo": "secciones", "en_orden": true,
 "lista": ["Introducción", "Metodología", "Resultados", "Discusión", "Conclusiones"]}
{"id": "A04", "tipo": "palabras_max", "valor": 1200}
{"id": "A05", "tipo": "cifra_presente", "verdad_py": "verdad = 569", "tolerancia": 0}
{"id": "A08", "tipo": "procedencia", "minimo": 0.85}
```

> Si tu solver aprueba todo tu golden set, el golden set no informa nada. Su diseño se califica.

2.b — Solver completo contra una versión recortada (10 %)

```
python evaluar_solver.py --solver tu_modulo:TuSolver --ruta <carpeta>
```

Córrelo sobre **tu solver completo** y sobre **una versión recortada**, con el mismo golden set y el mismo modelo:

- **sin grafo:** el investigador solo recibe los *k* fragmentos más parecidos (como en la 0.b), o

- **sin revisor:** un solo intento por script y ninguna revisión.

Reporta por tarea y para las dos versiones: **comprobaciones aprobadas**, **procedencia**, subtareas fallidas, **intentos de código**, tokens de entrada y salida **por agente**, y duración. Las tablas salen de `resultados_solver.csv` y `resumen_solver.csv`.

2.c — Los tres peores casos (10 %)

Para las **tres comprobaciones peores**, di qué agente falló —lector, grafo, plan, investigador, programador, sandbox, revisor o redactor— **usando la traza**: qué recibió, qué produjo y qué decidió el código.

Parte 3 — Frenos (10 %)

Implementa **cuatro frenos** y demuestra cada uno **haciéndolo saltar**, con su traza. Para no gastar, puedes forzarlos con un modelo de guion.

- **Presupuesto de tokens** por corrida, revisado **antes** de cada llamada, con una **reserva para el redactor**: si se acaba, el solver entrega lo que tiene y dice qué faltó.
- **Máximo de intentos por subtarea y detector de repetición:** al llegar al máximo, la subtarea queda fallida y se sigue con la siguiente; un script idéntico a uno rechazado no se vuelve a ejecutar.
- **Tiempo máximo del sandbox**, que mata todos los procesos del script.
- **Confirmación humana antes de usar la red:** un script que quiere descargar datos (`fetch_openml`, `load_dataset`, `download=True`) no corre sin aprobación de una persona.

Parte 4 — Una mejora (5 %)

Elige **una** después de medir el solver base, y mídela con el mismo golden set.

- **A — Subtareas en paralelo:** las independientes corren a la vez (API `Send` de LangGraph), repartidas entre las dos réplicas de la H200. Mide tiempo y tokens.
- **B — Herramientas por MCP:** `ejecutar_codigo` y `buscar_en_grafo` detrás de un servidor con `tools/list` y `tools/call`. Prueba: añadir una herramienta sin tocar los agentes.
- **C — Memoria entre tareas:** cada script aprobado entra al grafo y la tarea siguiente lo puede reutilizar. Compara intentos y tokens con y sin memoria.
- **D — Persona en el bucle:** un nodo con `interrupt` y *checkpointer* antes de ejecutar lo que usa red o tarda mucho, y antes de publicar el entregable.
- **E — Juez de otra familia:** que `gemma3` (Ollama de la H200) califique el reporte con la rúbrica del enunciado, y compara con tus comprobaciones: acuerdo y kappa.

Parte 5 — Preguntas (5 %)

Responde en 100–150 palabras cada una:

1. Hay un planificador que propone, un revisor que aprueba y una comprobación que publica. ¿Quién decide que la tarea está resuelta? Señálalo en tu código.
2. Con tu `resumen_solver.csv`: ¿qué agente gasta más tokens y por qué? ¿Qué ganó el grafo frente a la versión recortada, en comprobaciones y en tokens? Usa tus cifras.
3. ¿Qué haría falta para usar tu solver de forma honesta (qué se declara, qué evidencia se adjunta)? ¿Qué daño podría causar el código generado si el sandbox fallara?

Presupuesto

Con la H200 cuesta **cero**, pero tarda: en la solución de referencia cada tarea de práctica tomó entre 4 y 13 minutos y entre 79 000 y 138 000 tokens. La primera corrida suma unos 6 minutos para indexar las notas del curso (luego quedan guardadas). Indexa en paralelo. Para la Parte 4, **prueba con una tarea antes de correr todas**.

Qué entregas

- **Informe en PDF:** las tres salidas de la Parte 0; la arquitectura con las seis reglas; el grafo de la Tarea A; el modelo y la fecha; el golden set comentado; la tabla de la 2.b; los tres peores casos; los cuatro frenos activados; la mejora medida; la Parte 5.
- **Repositorio:** código, `requirements.txt`, tu `golden_tareas.json` y las corridas (`corridas/tarea-*/` con `traza.jsonl`, `plan.json` y el entregable).
- **`resultados_solver.csv` y `resumen_solver.csv`** de las dos versiones. Las tablas del informe salen de estos archivos.

> **Credenciales.** Ninguna clave de API, token o contraseña puede aparecer en el informe, el código, una traza, un script generado ni el repositorio. Guárdalas en un `.env` local que esté en `.gitignore`. **Ojo con el sandbox:** si el script hereda tus variables de entorno, el código que escriba el modelo puede imprimir tu clave en un log. Una clave expuesta deja el taller sin calificar.

Cómo se califica

Criterio	Peso
Parte 0 — Los tres problemas reproducidos, con salida y explicación	10 %
Parte 1 — Solver con los roles mínimos, las seis reglas, el formato pedido y el contrato	35 %
Parte 2.a — Golden set con dos tareas reales o más, verdad calculada, una trampa	10 %
Parte 2.b — Solver completo contra versión recortada: comprobaciones, procedencia, intentos, tokens por agente	10 %
Parte 2.c — Tres peores comprobaciones con el agente señalado y la traza	10 %
Parte 3 — Cuatro frenos, cada uno activado y con su traza	10 %
Parte 4 — Mejora medida contra el solver base	5 %
Parte 5 — Preguntas	5 %
Reproducibilidad: repositorio, versiones, CSV, corridas con traza y ninguna credencial	5 %

Un reporte con cifras sin origen, o tablas que no salen del CSV, pierden esos puntos aunque se lean bien.